

Redis trong Golang

Hướng Dẫn Đầy Đủ & Chi Tiết

go-redis/v9 • Redis 6.x/7.x • Golang 1.18+

1. Tổng Quan Redis

Redis (Remote Dictionary Server) là in-memory data store hỗ trợ nhiều kiểu dữ liệu, dùng phổ biến làm cache, message broker, session store và distributed lock.

Kiểu Dữ Liệu	Mô Tả	Use Case Phổ Biến
String	Chuỗi nhị phân, tối đa 512MB	Cache, counter, token, flag
Hash	Map field→value trong một key	Object/struct, user profile
List	Danh sách có thứ tự, push/pop hai đầu	Queue, stack, activity feed
Set	Tập hợp unique, không có thứ tự	Tags, friend list, dedup
Sorted Set	Set có score thực đính kèm	Leaderboard, ranking, timeseries
Bitmap	Mảng bit trên String	Feature flag, presence tracking
HyperLogLog	Đếm unique gần đúng, ~12KB cố định	Unique visitors, UV count
Stream	Log bất biến, hỗ trợ consumer group	Event log, message queue

2. Cài Đặt

```
# Cài go-redis v9
go get github.com/redis/go-redis/v9

# (Tùy chọn) Cài miniredis để test
go get github.com/alicebob/miniredis/v2
```

3. Kết Nối Redis

3.1 Kết Nối Đơn Giản (Standalone)

```
package main

import (
    "context"
    "fmt"
    "log"
    "github.com/redis/go-redis/v9"
)

var ctx = context.Background()

func main() {
    rdb := redis.NewClient(&redis.Options{
        Addr:     "localhost:6379",
        Password: "", // để trống nếu không có password
        DB:       0,   // database mặc định (0-15)
    })

    pong, err := rdb.Ping(ctx).Result()
    if err != nil {
        log.Fatalf("Không thể kết nối: %v", err)
    }
    fmt.Println("Kết nối thành công:", pong) // PONG
}
```

3.2 Kết Nối với URL

```
opt, err := redis.ParseURL("redis://:password@localhost:6379/0")
if err != nil { panic(err) }
rdb := redis.NewClient(opt)
```

3.3 Kết Nối TLS (Production)

```
import "crypto/tls"

rdb := redis.NewClient(&redis.Options{
    Addr: "my-redis.example.com:6380",
    TLSConfig: &tls.Config{
        MinVersion: tls.VersionTLS12,
    },
})
```

4. Kiểu Dữ Liệu & Lệnh Cơ Bản

4.1 String

```
// SET / GET
rdb.Set(ctx, "username", "alice", 0) // 0 = không TTL
val, err := rdb.Get(ctx, "username").Result()
if err == redis.Nil {
    fmt.Println("Key không tồn tại")
}

// SETNX — chỉ set nếu key chưa tồn tại
ok, _ := rdb.SetNX(ctx, "lock", "1", 10*time.Second).Result()

// MSET / MGET — set/get nhiều key cùng lúc
```

```

rdb.MSet(ctx, "k1", "v1", "k2", "v2")
vals, _ := rdb.MGet(ctx, "k1", "k2", "k3").Result()
// vals[2] = nil nếu k3 không tồn tại

// INCR / INCRBY - tăng counter (atomic)
rdb.Set(ctx, "views", 0, 0)
rdb.Incr(ctx, "views")           // 1
rdb.IncrBy(ctx, "views", 10)     // 11
rdb.Decr(ctx, "views")          // 10
count, _ := rdb.Get(ctx, "views").Int64()

// GETSET - lấy giá trị cũ, ghi giá trị mới (atomic)
old, _ := rdb.GetSet(ctx, "username", "bob").Result()

// APPEND, STRLEN, GETRANGE
rdb.Append(ctx, "log", "2024-01-01 login\n")
length, _ := rdb.StrLen(ctx, "username").Result()
sub, _ := rdb.GetRange(ctx, "username", 0, 2).Result()

```

4.2 Hash

Hash lưu nhiều field-value trong một key — lý tưởng để biểu diễn struct/object.

```

// HSET - ghi một hoặc nhiều field
rdb.HSet(ctx, "user:1", "name", "Alice", "age", 30, "email", "alice@example.com")

// HGET / HMGET
name, _ := rdb.HGet(ctx, "user:1", "name").Result()
vals, _ := rdb.HMGet(ctx, "user:1", "name", "age").Result()

// HGETALL - đọc tất cả, Scan vào struct
type User struct {
    Name string `redis:"name"`
    Age  int    `redis:"age"`
    Email string `redis:"email"`
}
var user User
rdb.HGetAll(ctx, "user:1").Scan(&user)

// HDEL, HEXISTS, HLEN, HKEYS, HVALS
rdb.HDel(ctx, "user:1", "email")
exists, _ := rdb.HEXists(ctx, "user:1", "name").Result()
count, _ := rdb.HLen(ctx, "user:1").Result()
keys, _ := rdb.HKeys(ctx, "user:1").Result()

// HINCRBY - tăng giá trị số trong hash
rdb.HIncrBy(ctx, "user:1", "age", 1)

```

4.3 List

List là danh sách có thứ tự, hỗ trợ push/pop từ cả hai đầu. Dùng làm queue (RPUSH + BLPOP) hoặc stack (LPUSH + LPOP).

```

// RPUSH / LPUSH - thêm vào cuối / đầu
rdb.RPush(ctx, "tasks", "task1", "task2", "task3")
rdb.LPush(ctx, "tasks", "task0")

// LRANGE - đọc (0, -1 = tất cả)
tasks, _ := rdb.LRange(ctx, "tasks", 0, -1).Result()

// LPOP / RPOP - lấy và xóa
first, _ := rdb.LPop(ctx, "tasks").Result()
last, _ := rdb.RPop(ctx, "tasks").Result()

// BLPOP - blocking pop (queue consumer)

```

```
result, err := rdb.BLPop(ctx, 5*time.Second, "tasks").Result()
// result[0] = tên list, result[1] = giá trị

// LLEN, LINDEX, LSET, LREM, LTRIM
rdb.LLen(ctx, "tasks")
rdb.LIndex(ctx, "tasks", 0)
rdb.LSet(ctx, "tasks", 0, "updated")
rdb.LRem(ctx, "tasks", 1, "task1") // xóa 1 lần xuất hiện từ đầu
rdb.LTrim(ctx, "tasks", 0, 99)    // giữ 100 phần tử đầu
```

4.4 Set

Set là tập hợp các giá trị không trùng lặp, không có thứ tự. Hỗ trợ phép toán tập hợp (union, intersect, diff).

```
// SADD, SMEMBERS, SCARD, SISMEMBER
rdb.SAdd(ctx, "tags", "golang", "redis", "backend")
members, _ := rdb.SMembers(ctx, "tags").Result()
count, _ := rdb.SCard(ctx, "tags").Result()
isMember, _ := rdb.SIsMember(ctx, "tags", "golang").Result()

// SPOP — lấy ngẫu nhiên và xóa
random, _ := rdb.SPop(ctx, "tags").Result()

// SRANDMEMBER — lấy ngẫu nhiên (không xóa)
rands, _ := rdb.SRandMemberN(ctx, "tags", 2).Result()

// Phép toán tập hợp
rdb.SAdd(ctx, "set1", "a", "b", "c")
rdb.SAdd(ctx, "set2", "b", "c", "d")
union, _ := rdb.SUnion(ctx, "set1", "set2").Result() // a,b,c,d
inter, _ := rdb.SInter(ctx, "set1", "set2").Result() // b,c
diff, _ := rdb.SDiff(ctx, "set1", "set2").Result()   // a

// Lưu kết quả vào key mới
rdb.SUnionStore(ctx, "result", "set1", "set2")
```

4.5 Sorted Set (ZSet)

Sorted Set giống Set nhưng mỗi phần tử có score (số thực). Redis tự động sắp xếp theo score — lý tưởng cho leaderboard, ranking.

```
// ZADD — thêm phần tử kèm score
rdb.ZAdd(ctx, "leaderboard",
    redis.Z{Score: 100, Member: "alice"},
    redis.Z{Score: 200, Member: "bob"},
    redis.Z{Score: 150, Member: "charlie"},
)

// ZRANGE — tăng dần; ZREVRANGE — giảm dần
members, _ := rdb.ZRange(ctx, "leaderboard", 0, -1).Result()
rdb.ZRevRange(ctx, "leaderboard", 0, -1)

// ZRANGE với score
withScores, _ := rdb.ZRangeWithScores(ctx, "leaderboard", 0, -1).Result()
for _, z := range withScores {
    fmt.Printf("%s: %.0f\n", z.Member, z.Score)
}

// ZSCORE, ZRANK, ZREVRANK
score, _ := rdb.ZScore(ctx, "leaderboard", "alice").Result()
rank, _ := rdb.ZRank(ctx, "leaderboard", "alice").Result()

// ZINCRBY — tăng score
```

```
rdb.ZIncrBy(ctx, "leaderboard", 50, "alice")

// ZRANGEBYSCORE - lọc theo khoảng score
members2, _ := rdb.ZRangeByScore(ctx, "leaderboard", &redis.ZRangeBy{
    Min: "100", Max: "200",
}).Result()

// ZREM, ZREMRANGEBYSCORE, ZREMRANGEBYRANK
rdb.ZRem(ctx, "leaderboard", "charlie")
rdb.ZRemRangeByScore(ctx, "leaderboard", "-inf", "99")
```

4.6 Bitmap

```
// SETBIT / GETBIT
rdb.SetBit(ctx, "user_active:2024-01-01", 1001, 1) // user 1001 active
bit, _ := rdb.GetBit(ctx, "user_active:2024-01-01", 1001).Result()

// BITCOUNT - đếm số bit = 1
count, _ := rdb.BitCount(ctx, "user_active:2024-01-01", nil).Result()

// BITOP - phép toán AND/OR/XOR giữa các bitmap
rdb.BitOpAnd(ctx, "result", "day1", "day2") // active cả 2 ngày
rdb.BitOpOr(ctx, "result", "day1", "day2")  // active ít nhất 1 ngày
```

4.7 HyperLogLog

Đếm số lượng unique items với độ chính xác ~0.81%, tốn cố định ~12KB bộ nhớ.

```
// PFADD - thêm phần tử
rdb.PFAdd(ctx, "unique_visitors", "user1", "user2", "user3", "user1")

// PFCOUNT - đếm unique (gần đúng)
count, _ := rdb.PFCount(ctx, "unique_visitors").Result() // ≈ 3

// PFMERGE - merge nhiều HLL
rdb.PFMerge(ctx, "total", "visitors jan", "visitors feb")
```

4.8 Stream

Stream là log/queue bất biến, hỗ trợ consumer group — phù hợp làm event sourcing hoặc message queue.

```
// XADD - thêm message
id, _ := rdb.XAdd(ctx, &redis.XAddArgs{
    Stream: "events",
    Values: map[string]interface{}{"action": "login", "user": "alice"},
}).Result()

// XRANGE - đọc messages
messages, _ := rdb.XRange(ctx, "events", "-", "+").Result()
for _, msg := range messages {
    fmt.Println(msg.ID, msg.Values)
}

// Consumer Group
rdb.XGroupCreate(ctx, "events", "mygroup", "0")

msgs, _ := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
    Group: "mygroup", Consumer: "consumer1",
    Streams: []string{"events", ">"}, Count: 10,
```

```

}).Result()

// XACK - xác nhận đã xử lý
for _, stream := range msgs {
    for _, msg := range stream.Messages {
        // xử lý msg...
        rdb.XAck(ctx, "events", "mygroup", msg.ID)
    }
}

```

5. TTL & Expiration

```

// Set với TTL ngay khi tạo
rdb.Set(ctx, "session:abc", "data", 30*time.Minute)

// EXPIRE - đặt TTL cho key đã tồn tại
rdb.Expire(ctx, "mykey", 60*time.Second)

// EXPIREAT - hết hạn tại thời điểm cụ thể
rdb.ExpireAt(ctx, "mykey", time.Now().Add(24*time.Hour))

// TTL - kiểm tra thời gian còn lại
ttl, _ := rdb.TTL(ctx, "mykey").Result()
// ttl == -1 → không có TTL
// ttl == -2 → key không tồn tại
// ttl > 0 → còn bao nhiêu giây

// PERSIST - xóa TTL, key tồn tại mãi mãi
rdb.Persist(ctx, "mykey")

// PEXPIRE / PTTL - TTL tính bằng millisecond
rdb.PExpire(ctx, "mykey", 500*time.Millisecond)
pttl, _ := rdb.PTTL(ctx, "mykey").Result()

```

6. Pipeline & Batch

Pipeline gửi nhiều lệnh cùng lúc trong một round-trip, giảm đáng kể độ trễ network khi cần thực thi nhiều lệnh liên tiếp.

```

// Pipeline thông thường (không atomic)
pipe := rdb.Pipeline()
incr := pipe.Incr(ctx, "counter")
pipe.Expire(ctx, "counter", time.Hour)
_, err := pipe.Exec(ctx)
fmt.Println(incr.Val()) // lấy kết quả sau Exec

// Pipelined - cách viết gọn hơn
var counter *redis.IntCmd
rdb.Pipelined(ctx, func(pipe redis.Pipeliner) error {
    counter = pipe.Incr(ctx, "counter")
    pipe.Expire(ctx, "counter", time.Hour)
    return nil
})
fmt.Println(counter.Val())

// TxPipeline - pipeline với MULTI/EXEC (atomic)
txPipe := rdb.TxPipeline()
txPipe.Set(ctx, "key1", "val1", 0)
txPipe.Set(ctx, "key2", "val2", 0)
_, err = txPipe.Exec(ctx)

```

7. Transaction (MULTI/EXEC)

Dùng WATCH + TxPipelined để thực hiện Optimistic Locking: nếu key bị thay đổi trước khi EXEC, transaction bị hủy (TxFailedErr) và cần thử lại.

```
const maxRetries = 3

for i := 0; i < maxRetries; i++ {
    err := rdb.Watch(ctx, func(tx *redis.Tx) error {
        // Đọc giá trị hiện tại
        n, err := tx.Get(ctx, "balance").Int()
        if err != nil && err != redis.Nil { return err }

        newBalance := n - 100
        if newBalance < 0 { return fmt.Errorf("số dư không đủ") }

        // TxPipelined: atomic nếu key chưa thay đổi
        _, err = tx.TxPipelined(ctx, func(pipe redis.Pipeliner) error {
            pipe.Set(ctx, "balance", newBalance, 0)
            return nil
        })
        return err
    }, "balance") // watch key "balance"

    if err == nil { break } // thành công
    if err == redis.TxFailedErr { continue } // race condition → retry
    log.Fatal(err)
}
```

8. Pub/Sub

Publisher

```
err := rdb.Publish(ctx, "channel:news", "Breaking news!").Err()
```

Subscriber

```
// Subscribe một hoặc nhiều channel
pubsub := rdb.Subscribe(ctx, "channel:news", "channel:sports")
defer pubsub.Close()

// Subscribe theo pattern
pubsub = rdb.PSubscribe(ctx, "channel:*")

// Nhận message qua channel (blocking)
ch := pubsub.Channel()
for msg := range ch {
    fmt.Printf("Channel: %s | Payload: %s\n", msg.Channel, msg.Payload)
}

// Nhận với timeout
msg, err := pubsub.ReceiveTimeout(ctx, 5*time.Second)
switch m := msg.(type) {
case *redis.Message:
    fmt.Println("Message:", m.Payload)
case *redis.Subscription:
    fmt.Println("Subscribed to:", m.Channel)
}
```

9. Lua Scripting

Lua script chạy atomic trên Redis server — không bị interrupt, đảm bảo tính nhất quán cho các thao tác phức tạp.

```
// Script: tăng counter nếu chưa vượt quá giới hạn
script := redis.NewScript(`
    local current = redis.call("GET", KEYS[1])
    if current == false then
        redis.call("SET", KEYS[1], 1)
        redis.call("EXPIRE", KEYS[1], ARGV[1])
        return 1
    elseif tonumber(current) < tonumber(ARGV[2]) then
        return redis.call("INCR", KEYS[1])
    else
        return 0
    end
`)

result, err := script.Run(ctx, rdb,
    []string{"rate:user:1"}, // KEYS
    60, 10,                  // ARGV: TTL=60s, limit=10
).Int()

if result == 0 {
    fmt.Println("Rate limit exceeded")
} else {
    fmt.Printf("Request #%d accepted\n", result)
}
```

10. Redis Cluster

```
rdb := redis.NewClusterClient(&redis.ClusterOptions{
    Addrs: []string{
        "localhost:7000", "localhost:7001", "localhost:7002",
        "localhost:7003", "localhost:7004", "localhost:7005",
    },
    Password: "secret",
    RouteByLatency: true,
})

// ForEachShard — chạy lệnh trên tất cả shard
rdb.ForEachShard(ctx, func(ctx context.Context, shard *redis.Client) error {
    return shard.FlushDB(ctx).Err()
})

// HashTag — đảm bảo các key cùng slot
rdb.Set(ctx, "{user:1}.profile", "data1", 0)
rdb.Set(ctx, "{user:1}.settings", "data2", 0)
// Cả 2 key cùng slot vì cùng hash tag {user:1}
```

11. Redis Sentinel (High Availability)

```
rdb := redis.NewFailoverClient(&redis.FailoverOptions{
    MasterName: "mymaster",
```



```

    SentinelAddrs: []string{
        "sentinel1:26379",
        "sentinel2:26379",
        "sentinel3:26379",
    },
    Password:      "redis-password",
    SentinelPassword: "sentinel-password",
    DB:            0,
})

```

12. Connection Pool

```

rdb := redis.NewClient(&redis.Options{
    Addr: "localhost:6379",

    PoolSize:      20,           // max số connection
    MinIdleConns:  5,           // min connection luôn duy trì
    MaxIdleConns:  10,          // max connection idle
    PoolTimeout:   30 * time.Second, // thời gian chờ lấy connection
    ConnMaxIdleTime: 5 * time.Minute, // idle quá lâu sẽ bị đóng
    ConnMaxLifetime: 1 * time.Hour, // connection sống tối đa
})

// Kiểm tra trạng thái pool
stats := rdb.PoolStats()
fmt.Printf("Hits: %d | Misses: %d | Timeouts: %d\n",
    stats.Hits, stats.Misses, stats.Timeouts)
fmt.Printf("Total: %d | Idle: %d | Stale: %d\n",
    stats.TotalConns, stats.IdleConns, stats.StaleConns)

```

13. Context & Timeout

```

// Timeout cho từng lệnh
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()
val, err := rdb.Get(ctx, "mykey").Result()

// Timeout toàn cục khi tạo client
rdb := redis.NewClient(&redis.Options{
    Addr:      "localhost:6379",
    DialTimeout: 5 * time.Second,
    ReadTimeout: 3 * time.Second,
    WriteTimeout: 3 * time.Second,
})

// Truyền context qua function
func fetchData(ctx context.Context, rdb *redis.Client, key string) (string, error) {
    return rdb.Get(ctx, key).Result()
}

```

14. Scan (Duyệt Key An Toàn)

❑ **Lưu ý:** Không dùng KEYS * trong production vì nó blocking. Thay bằng SCAN để duyệt không chặn.

```
// Scan tất cả key với Iterator (cách gọn nhất)
iter := rdb.Scan(ctx, 0, "*", 0).Iterator()
for iter.Next(ctx) {
    fmt.Println("Key:", iter.Val())
}
if err := iter.Err(); err != nil { panic(err) }

// Scan với pattern
iter = rdb.Scan(ctx, 0, "user:*", 100).Iterator()
for iter.Next(ctx) { fmt.Println(iter.Val()) }

// Scan thủ công (kiểm soát từng batch)
var cursor uint64
for {
    var batch []string
    batch, cursor, _ = rdb.Scan(ctx, cursor, "*", 100).Result()
    // xử lý batch...
    if cursor == 0 { break }
}

// HScan / SScan / ZScan - tương tự cho Hash, Set, ZSet
hIter := rdb.HScan(ctx, "highash", 0, "*", 100).Iterator()
for hIter.Next(ctx) { fmt.Println(hIter.Val()) }
```

15. Cache Pattern (Best Practices)

Cache-Aside (Lazy Loading)

Pattern phổ biến nhất: đọc từ cache trước, nếu miss thì query DB rồi lưu vào cache.

```
func GetUser(ctx context.Context, rdb *redis.Client, db *sql.DB, userID int) (*User, error) {
    key := fmt.Sprintf("user:%d", userID)

    // 1. Thử lấy từ cache
    val, err := rdb.Get(ctx, key).Bytes()
    if err == nil {
        var user User
        if err := json.Unmarshal(val, &user); err == nil {
            return &user, nil // cache hit
        }
    }

    // 2. Cache miss → query DB
    var user User
    row := db.QueryRowContext(ctx, "SELECT id, name FROM users WHERE id = ?",
        userID)
    if err := row.Scan(&user.ID, &user.Name); err != nil { return nil, err }

    // 3. Lưu vào cache
    data, _ := json.Marshal(user)
    rdb.Set(ctx, key, data, 10*time.Minute)
    return &user, nil
}

// Invalidate cache khi update
func UpdateUser(ctx context.Context, rdb *redis.Client, db *sql.DB, user *User) error {
    _, err := db.ExecContext(ctx, "UPDATE users SET name=? WHERE id=?", user.Name,
        user.ID)
    if err != nil { return err }
    return rdb.Del(ctx, fmt.Sprintf("user:%d", user.ID)).Err()
}
```

16. Distributed Lock

Dùng SET NX EX để acquire lock. Dùng Lua script để release an toàn (chỉ xóa lock của chính mình).

```
type DistributedLock struct {
    client *redis.Client
    key, value string
    ttl time.Duration
}

func NewLock(client *redis.Client, key string, ttl time.Duration) *DistributedLock {
    return &DistributedLock{
        client: client, key: key,
        value: fmt.Sprintf("%d", time.Now().UnixNano()), // unique
        ttl: ttl,
    }
}

func (l *DistributedLock) Acquire(ctx context.Context) (bool, error) {
    return l.client.SetNX(ctx, l.key, l.value, l.ttl).Result()
}

// Lua script: chỉ xóa nếu value khớp (tránh xóa nhầm lock của process khác)
var unlockScript = redis.NewScript(`
    if redis.call("GET", KEYS[1]) == ARGV[1] then
        return redis.call("DEL", KEYS[1])
    else return 0 end
`)

func (l *DistributedLock) Release(ctx context.Context) error {
    , err := unlockScript.Run(ctx, l.client, []string{l.key}, l.value).Result()
    return err
}

// Cách dùng
lock := NewLock(rdb, "job:process:lock", 30*time.Second)
acquired, _ := lock.Acquire(ctx)
if !acquired { return fmt.Errorf("đang bị lock") }
defer lock.Release(ctx)
// xử lý an toàn...
```

17. Rate Limiting

Fixed Window

```
func IsAllowed(ctx context.Context, rdb *redis.Client, userID string, limit int)
(bool, error) {
    // Window 1 phút: key thay đổi mỗi phút
    key := fmt.Sprintf("ratelimit:%s:%d", userID, time.Now().Unix()/60)

    pipe := rdb.Pipeline()
    incr := pipe.Incr(ctx, key)
    pipe.Expire(ctx, key, time.Minute)
    if , err := pipe.Exec(ctx); err != nil { return false, err }

    return incr.Val() <= int64(limit), nil
}
```

Sliding Window (dùng Sorted Set)

```
func SlidingWindowAllow(ctx context.Context, rdb *redis.Client,
    key string, limit int, window time.Duration) (bool, error) {

    now := time.Now()
    windowStart := now.Add(-window)

    pipe := rdb.TxPipeline()
    // Xóa requests cũ ngoài window
    pipe.ZRemRangeByScore(ctx, key, "-inf",
        strconv.FormatInt(windowStart.UnixMilli(), 10))
    // Thêm request hiện tại
    pipe.ZAdd(ctx, key, redis.Z{
        Score: float64(now.UnixMilli()),
        Member: strconv.FormatInt(now.UnixNano(), 10),
    })
    count := pipe.ZCard(ctx, key)
    pipe.Expire(ctx, key, window)
    if , err := pipe.Exec(ctx); err != nil { return false, err }

    return count.Val() <= int64(limit), nil
}
```

18. Session Management

```
type SessionStore struct {
    rdb *redis.Client
    ttl time.Duration
}

func (s *SessionStore) Set(ctx context.Context, sessionID string,
    data map[string]interface{}) error {
    encoded, _ := json.Marshal(data)
    return s.rdb.Set(ctx, "session:"+sessionID, encoded, s.ttl).Err()
}

func (s *SessionStore) Get(ctx context.Context, sessionID string)
    (map[string]interface{}, error) {
    val, err := s.rdb.Get(ctx, "session:"+sessionID).Bytes()
    if err == redis.Nil { return nil, nil } // session không tồn tại
    if err != nil { return nil, err }

    var data map[string]interface{}
    json.Unmarshal(val, &data)

    // Sliding expiration: gia hạn mỗi lần access
    s.rdb.Expire(ctx, "session:"+sessionID, s.ttl)
    return data, nil
}

func (s *SessionStore) Delete(ctx context.Context, sessionID string) error {
    return s.rdb.Del(ctx, "session:"+sessionID).Err()
}
```

19. Xử Lý Lỗi

```
val, err := rdb.Get(ctx, "mykey").Result()

switch {
case err == redis.Nil:
    // Key không tồn tại — KHÔNG phải lỗi thực sự
    fmt.Println("Key không tồn tại")

case err != nil:
```

```
// Lỗi thực sự: mất kết nối, timeout, sai type...
log.Printf("Redis error: %v", err)

default:
    fmt.Println("Value:", val)
}

// Tự động retry khi lỗi kết nối
rdb := redis.NewClient(&redis.Options{
    Addr:      "localhost:6379",
    MaxRetries: 3,
    MinRetryBackoff: 100 * time.Millisecond,
    MaxRetryBackoff: 1 * time.Second,
})
```

20. Test với miniredis

miniredis chạy Redis in-memory trong process Go, không cần Redis server thật. Hỗ trợ fast-forward time để test TTL.

```
package mypackage test

import (
    "context"
    "testing"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
    "github.com/stretchr/testify/assert"
)

func setupTestRedis(t *testing.T) (*redis.Client, *miniredis.Miniredis) {
    t.Helper()
    mr, err := miniredis.Run()
    if err != nil { t.Fatalf("miniredis: %v", err) }
    client := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    return client, mr
}

func TestGetCached(t *testing.T) {
    ctx := context.Background()
    rdb, mr := setupTestRedis(t)
    defer mr.Close(); defer rdb.Close()

    rdb.Set(ctx, "user:1", `{"id":1,"name":"Alice"}`, 0)

    val, err := rdb.Get(ctx, "user:1").Result()
    assert.NoError(t, err)
    assert.Contains(t, val, "Alice")
}

func TestTTLExpiry(t *testing.T) {
    ctx := context.Background()
    rdb, mr := setupTestRedis(t)
    defer mr.Close()

    rdb.Set(ctx, "tempkey", "value", 5*time.Second)

    // Fast-forward thời gian với miniredis
    mr.FastForward(6 * time.Second)

    _, err := rdb.Get(ctx, "tempkey").Result()
    assert.Equal(t, redis.Nil, err) // key đã hết hạn
}
```

Tóm Tắt Nhanh

Tính Năng	Hàm / Phương Pháp Chính
String cơ bản	Set, Get, Incr, MSet, MGet, GetSet, SetNX
Hash	HSet, HGet, HGetAll, HDel, HExists, HIncrBy
List (queue/stack)	RPush, LPush, LPop, RPop, BLPop, LRange, LTrim
Set (unique)	SAdd, SMembers, SCard, SInter, SUnion, SDiff
Sorted Set	ZAdd, ZRange, ZScore, ZRank, ZIncrBy, ZRangeByScore
TTL	Expire, ExpireAt, TTL, PTTL, Persist
Batch (nhanh hơn)	Pipeline, Pipelined, TxPipeline
Atomic Transaction	Watch + TxPipelined (Optimistic Lock)
Pub/Sub	Publish, Subscribe, PSubscribe
Lua Script	redis.NewScript + .Run
Distributed Lock	SetNX + Lua unlock script
Rate Limiting	Incr+Expire (Fixed) / ZAdd+ZCard (Sliding)
Session	Set/Get/Del + Expire (sliding TTL)
Scan an toàn	Scan().Iterator() — không dùng KEYS *
Cluster	NewClusterClient + HashTag {key}
Sentinel (HA)	NewFailoverClient
Testing	miniredis.Run() + FastForward